

# 分布式训练与大模型系统

## 目 录

|                                        |    |
|----------------------------------------|----|
| 1 引言：为什么单卡已经不够了 .....                  | 3  |
| 2 并行范式总览：从 3D 到 6D .....               | 3  |
| 3 集合通信基础 .....                         | 3  |
| 3.1 六个基本原语 .....                       | 4  |
| 3.2 通信代价模型 $\alpha + n\beta$ .....     | 4  |
| 3.3 Ring 算法：大消息的利器 .....               | 4  |
| 3.4 通信与计算重叠 .....                      | 4  |
| 4 数据并行（DP） .....                       | 5  |
| 4.1 最简单的分布式训练 .....                    | 5  |
| 4.2 参数服务器 vs Allreduce .....           | 5  |
| 4.3 ZeRO：榨干 DP 的显存 .....               | 5  |
| 4.4 混合精度与优化器状态 .....                   | 6  |
| 4.5 DP 的极限 .....                       | 6  |
| 5 流水线并行（PP） .....                      | 6  |
| 5.1 朴素流水线与关键指标 .....                   | 6  |
| 5.2 GPipe：切微批次 .....                   | 6  |
| 5.3 1F1B：尽早反向释放显存 .....                | 6  |
| 6 张量并行（TP） .....                       | 7  |
| 6.1 矩阵乘的多种切分 .....                     | 7  |
| 6.2 Megatron-LM 的 MLP .....            | 7  |
| 6.3 Megatron-LM 的 Self-Attention ..... | 7  |
| 7 上下文并行与专家并行 .....                     | 8  |
| 7.1 上下文并行（CP） .....                    | 8  |
| 7.2 专家并行（EP）与 MoE .....                | 8  |
| 8 6D 并行与自动并行 .....                     | 8  |
| 9 Transformer 定量分析 .....               | 8  |
| 9.1 注意力与多头注意力 .....                    | 8  |
| 9.2 参数量与 FLOPs .....                   | 9  |
| 9.3 显存估算与梯度检查点 .....                   | 9  |
| 10 大模型推理系统 .....                       | 9  |
| 10.1 推理的两阶段 .....                      | 9  |
| 10.2 KV Cache .....                    | 9  |
| 10.3 连续批处理 .....                       | 10 |

|                                        |           |
|----------------------------------------|-----------|
| 10.4 Prefill/Decode 分离部署 .....         | 10        |
| 10.5 Paged Attention .....             | 10        |
| <b>11 Flash Attention .....</b>        | <b>10</b> |
| 11.1 动机: $S$ 维爆炸 .....                 | 10        |
| <b>12 前沿与总结 .....</b>                  | <b>10</b> |
| 12.1 投机解码 .....                        | 10        |
| 12.2 DeepSeek-V4: CSA + HCA 双流混合 ..... | 11        |
| 12.3 硬件视角: 放什么在哪 .....                 | 11        |
| <b>13 本章你将学会 .....</b>                 | <b>11</b> |
| <b>14 要点速查 .....</b>                   | <b>11</b> |
| <b>15 小结 .....</b>                     | <b>12</b> |

# 1 引言：为什么单卡已经不够了

摩尔定律正在失效，单卡 GPU 的算力和显存增长放缓。但另一边，模型参数量却每 4 到 6 个月就翻倍：从 GPT-2（15 亿参数）到 GPT-3（1750 亿参数），再到 GPT-4 与 DeepSeek-V3（6710 亿参数）。显存需求的增长远远快于单卡显存的扩容。

结论很残酷：单张 GPU 已经放不下一个现代大模型。要想训练、甚至推理这些模型，分布式并行是唯一的出路。本章系统地讲解如何把一个大模型切分到成百上千张 GPU 上，以及大模型推理系统如何高效服务用户。

**直觉** 你可以把单卡训练想象成“一个人扛一整头牛”，牛大到扛不动时只有两条路：要么把牛切成块分给一群人扛（切分模型），要么让一群人各自扛一份牛肉再汇总（并行数据）。大模型训练正是这两种思路的组合，而且切分维度多达六个。

本章分两大部分：第一部分讲分布式训练（Distributed Training）的六种并行维度与通信基础；第二部分讲大模型系统（LLM Systems），包括 Transformer 的定量分析、推理系统、Flash Attention 与前沿进展。

## 2 并行范式总览：从 3D 到 6D

现代大模型训练组合了多达六个并行维度，前三个是经典的 3D，后三个是扩展：

1. **DP**（Data Parallelism，数据并行）：切分输入 batch，每张卡持有完整模型副本。
2. **PP**（Pipeline Parallelism，流水线并行）：把层切到不同卡上（竖切，算子间）。
3. **TP**（Tensor Parallelism，张量并行）：在层内切分张量（横切，算子内）。
4. **SP**（Sequence Parallelism，序列并行）：沿序列切分 LayerNorm/Dropout 的激活。
5. **CP**（Context Parallelism，上下文并行）：沿序列切分 KV/attention（长上下文）。
6. **EP**（Expert Parallelism，专家并行）：把 MoE 专家切到不同卡上。

记忆口诀：**算子间**（Inter-op）切的是算子之间，对应 PP；**算子内**（Intra-op）切的是算子内部的张量维度，对应 DP/TP/SP/CP/EP。经典 3D 是 DP·TP·PP，加上 SP·CP·EP 就是 6D。

这两个概念是正交的：数据并行虽然名字像“算子间”，但它切的是 batch 这一**张量维度**，所以其实属于算子内并行。算子间并行只需相邻 stage 间点对点通信（P2P），算子内并行必须同步整个张量（集合通信 Collective）。

## 3 集合通信基础

并行训练离不开通信。先认清两种通信模型：**P2P**（Point-to-Point，点对点）只有两个进程直接交换消息；**Collective**（集合通信）由多个进程共同参与。集合通信代价通常远高于 P2P，因为要等所有参与者到齐，而且任何一个进程卡住都会阻塞整组。

**直觉** 通信原语像积木：算子间只需相邻 stage 传递，所以用 P2P；算子内每次都要同步整块张量，所以用集合通信。主流实现是 NVIDIA 的 NCCL、MPI 和 Gloo。

### 3.1 六个基本原语

集合通信有六个基本原语，它们两两互为逆操作：

1. **Broadcast** (广播)：单节点数据发给所有节点。
2. **Reduce** (规约，到一)：把所有节点的值聚合（求和/最大/最小）到一个节点。
3. **Scatter** (散射)：单节点数据切成等份发给不同节点。
4. **Gather** (收集)：把多节点数据收到一个节点。
5. **Allgather** (全收集)：每个节点最终都拿到所有片段，等于 Gather + Broadcast。
6. **Allreduce** (全规约)：每个节点最终都拿到求和结果，等于 Reduce + Broadcast。

最重要的恒等式：Allreduce = Reduce-Scatter + Allgather。这是 *Ring-Allreduce* 和 *ZeRO* 优化的数学基础。*Reduce-Scatter* 先规约再散射，让每个节点只保留求和结果的一个分片；*Allgather* 再把所有分片广播开。

### 3.2 通信代价模型 $\alpha + n\beta$

每次点对点通信的时间可以用 **LogP/Hockney 模型** 近似： $T = \alpha + n\beta$ ，其中  $\alpha$  是固定启动开销， $\beta = \frac{1}{B}$  ( $B$  是链路带宽)， $n$  是消息字节数。

**直觉** 小消息时  $\alpha \gg n\beta$ ，瓶颈在延迟，目标是减少通信跳数，典型算法是最小生成树 (MST)；大消息时  $\alpha \ll n\beta$ ，瓶颈在带宽，目标是最大化聚合带宽，典型算法是环算法 (Ring)。

实际中 NCCL 会根据消息大小和拓扑自动选择：小消息用 Tree/Chain，大消息用 Ring，超大消息用双向 Ring 或分层 Ring。

### 3.3 Ring 算法：大消息的利器

**Ring Allreduce** 把所有节点排成逻辑环，让消息沿环流动。 $P$  个节点、 $n$  字节的复杂度是  $T = 2(P-1)\alpha + 2 \cdot \frac{P-1}{P} \cdot n\beta$ 。

**例** 关键观察：带宽项  $\frac{P-1}{P} \cdot n\beta$  当  $P$  很大时趋近于  $n\beta$ ，与节点数  $P$  几乎无关。这意味着无论你用多少张 GPU，带宽都几乎被打满。这就是 Ring Allreduce 成为分布式训练主力算法的原因。

### 3.4 通信与计算重叠

反向传播的计算和梯度 Allreduce 可以并行：早算出的梯度可以立刻发出去。**PyTorch DDP** 把梯度按参数注册顺序分成若干 bucket (bucket\_cap\_mb=25)，一旦某个 bucket 的梯度都就绪就立即触发 Allreduce，与后续层的反向传播重叠。

没有重叠时，反向传播和 Allreduce 串行排队，GPU 大量空闲；重叠后两者交错进行，通信几乎被计算隐藏。这是所有并行优化的共同底色：让通信和计算重叠起来。

## 4 数据并行 (DP)

### 4.1 最简单的分布式训练

**数据并行**的前提是：单卡能放下整个模型（参数、梯度、优化器状态、激活）。每张卡持有完整模型副本，各自吃一份输入做前向和反向，算出本地梯度后 Allreduce 求平均，再用相同的梯度更新参数。

1. 每张 GPU 独立做前向 + 反向，产生本地梯度  $g^k$ 。
2. Allreduce:  $\hat{g} = \frac{1}{P} \sum_k g^k$ 。
3. 用  $\hat{g}$  更新参数（每张卡结果相同）。

### 4.2 参数服务器 vs Allreduce

早期的 DP 用 **参数服务器** (Parameter Server, PS)：每个 worker 算完梯度发给 PS，PS 聚合更新后再把新参数广播回去。它有两个致命问题：通信瓶颈（所有 worker 都和 PS 交互，PS 带宽随 worker 数线性增长）和单点故障（PS 挂了整个训练就停）。

**Allreduce**（去中心化 DP）让 worker 之间直接成环，无需中心节点。Uber 在 2017 年的 **Horovod** 把它引入 ML 训练，NVIDIA 在 NCCL 里做了 Ring Allreduce 优化，PyTorch DDP 和 TensorFlow MultiWorkerMirroredStrategy 都基于它。带宽可扩展性从  $\propto P$  变成常数，但容错性仍是整组失败。

### 4.3 ZeRO：榨干 DP 的显存

标准 DP 在每张卡上都存完整的  $\theta$ （参数）、 $g$ （梯度）和优化器状态（Adam 的  $m$ 、 $v$ ），造成严重冗余。**ZeRO** (Zero Redundancy Optimizer) 按级别逐步切分这些状态。设参数数量为  $\Psi$ ，DP 并行度  $N$ ，Adam 常数  $K \approx 12$  字节（fp32 master 参数 4 + momentum 4 + variance 4）：

1. **Baseline DP**：每卡  $\Psi(2 + 2 + K) = 16\Psi$  字节。
2. **ZeRO-1**（切优化器状态）： $2\Psi + 2\Psi + \frac{K\Psi}{N}$ 。
3. **ZeRO-2**（切梯度+优化器）： $2\Psi + \frac{(2+K)\Psi}{N}$ 。
4. **ZeRO-3**（切参数+梯度+优化器，即 FSDP）： $\frac{(2+2+K)\Psi}{N}$ 。

**例** 以 GPT-3 175B、 $N = 1024$  为例：Baseline 每卡约 2800 GB（根本放不下），ZeRO-1 约 702 GB，ZeRO-2 约 352 GB，ZeRO-3 约 2.7 GB。ZeRO-3 把显存压到原来的千分之一量级。

代价是通信量：ZeRO-1/2 的通信量等于 DDP（一次 Allreduce），几乎“免费”省显存；ZeRO-3 需要在前向/反向前 AllGather 参数（ $2\Psi$ ）再加反向时 Reduce-Scatter 梯度（ $\Psi$ ），总共约  $3\Psi$ ，是 DDP 的 3 倍，属于“用带宽换显存”。

**ZeRO-3/FSDP 的生命周期**：前向前 AllGather 收集参数分片得到完整参数，前向完立刻释放；反向前再次 AllGather，反向后 Reduce-Scatter 梯度让每卡只保留  $\frac{1}{P}$  的梯度。这可以和反向计算完全重叠（预取下一层的 AllGather）。PyTorch 官方实现是 **FullyShardedDataParallel (FSDP)**。

## 4.4 混合精度与优化器状态

**混合精度训练** (Micikevicius 2018) 用 FP16 做前向/反向计算, 用 loss scaling 防止梯度下溢; 同时保留 FP32 master 副本做优化器更新, 避免累积舍入误差。每个参数的总开销是 16 字节: FP16 参数 2 + FP16 梯度 2 + FP32 master 4 + FP32 Adam  $m$  4 + FP32 Adam  $v$  4。

趋势: **BF16** 不需要 loss scaling, 动态范围等于 FP32; **FP8** (H100) 由 Transformer Engine 逐层选择精度。无论精度怎么变, 主要算力仍来自 Self-Attention 和 FFN 的矩阵乘。

## 4.5 DP 的极限

DP 有两个硬约束: 一是显存, 即便 ZeRO-3 也要求单卡能放下某一层的完整前向/反向激活, 超大 attention 仍吃力; 二是 batch size, DP 只切 batch, 全局 batch 大到一定程度会损害泛化 (学习率和 warmup 要重调)。当模型放不下单卡, 就必须切分模型本身: 算子间走流水线并行, 算子内走张量并行。

## 5 流水线并行 (PP)

**流水线并行** (算子间并行) 把模型的计算图切成多个 stage, 每个 stage 放在不同设备上, 用流水线调度提升设备利用率。朴素做法的问题是: 下一 stage 必须等上一 stage 的结果, 产生大量气泡 (bubble); 反向还要暂存中间激活, 进一步加重显存压力。

### 5.1 朴素流水线与关键指标

衡量流水线有三个指标: **气泡率** (bubble ratio, 空闲时间占比)、**峰值显存** (peak memory) 和 **收敛性** (是否等价于单卡训练)。

### 5.2 GPipe: 切微批次

**GPipe** (Google 2019) 把输入 batch 切成  $N$  个微批次 (micro-batch), 每个微批次依次流过流水线, 梯度累积:  $\nabla L_{\theta(x)} = \frac{1}{N} \sum_i \nabla L_{\theta(x_i)}$ 。

**例** 取  $D = 4$  个 stage、 $N = 4$  个微批次。先做完 4 个微批次的前向 ( $F_0$  到  $F_3$ ), 再做 4 个反向。气泡率  $= \frac{D-1}{D-1+N} = \frac{3}{7} \approx 43\%$ 。微批次越多, 气泡率越低。

缺点: 必须保留所有微批次的激活直到反向阶段, 峰值显存  $O(N)$ , 随微批次数爆炸。

### 5.3 1F1B: 尽早反向释放显存

**1F1B** (PipeDream Flush) 的核心想法是尽早启动反向: 当前向到达最后一个 stage 时, 立刻触发反向, 释放激活。

**直觉** 关键改进: 峰值显存从 GPipe 的  $O(N)$  (随微批次数增长) 降到  $O(D)$  (只与 stage 数有关, 与微批次数无关)。这就像流水线上工件做完一步就清走, 不让半成品堆满车间。

流水线调度家族：GPipe  $\rightarrow$  1F1B  $\rightarrow$  Interleaved（交错） $\rightarrow$  TeraPipe（token 级） $\rightarrow$  Chimera（双向）。异步调度还有 AMPNet、PipeDream、PipeDream-2BW，但可能不等价于单卡训练。

## 6 张量并行（TP）

张量并行（算子内并行）的目标是切分单个算子（矩阵乘、卷积、LayerNorm）的张量维度，让多个设备协作完成同一个计算。难点在于：不同的切分方案通信代价不同，还要考虑相邻算子之间重新切分的通信成本（边代价）。

### 6.1 矩阵乘的多种切分

考虑  $C_{m \times k} = A_{m \times n} B_{n \times k}$ ，记  $S(i)$  为沿第  $i$  维切分、 $R$  为复制。有几种方案：

1. 切  $A$  的第 0 维 ( $A: S(0), B: R$ )：无需通信，每卡得  $C: S(0)$ 。
2. 切  $A$  的第 1 维 ( $A: S(1), B: S(0)$ )：各卡得部分和，需要 Allreduce。
3. 切  $A$  第 0 维、 $B$  第 1 维（部分分块）：无需通信， $C$  跨维度共存。

**直觉** 两种典型切分：**Type-1 列并行**（切  $B$  的列）无需通信；**Type-2 行并行**（切  $B$  的行）需要 Allreduce 聚合部分和。Megatron-LM 巧妙组合这两种切分。

### 6.2 Megatron-LM 的 MLP

Megatron-LM 把 MLP 的两个矩阵乘串成“列并行  $\rightarrow$  行并行”：

1. **列并行**  $Y = XA$ ：把  $A = [A_1, A_2]$  按列切， $Y_i = XA_i$ ，无需前期通信；GeLU 是逐元素的，可以本地算。
2. **行并行**  $Z = YB$ ：把  $B = [B_1; B_2]$  按行切、 $Y = [Y_1, Y_2]$ ，则  $Z = \sum_i Y_i B_i$  是部分和，需要一次 Allreduce。

妙处：列并行接行并行，中间张量不需要 AllGather，整个 MLP 只需 1 次 Allreduce（反向再加 1 次）。这把通信量压到最低。

### 6.3 Megatron-LM 的 Self-Attention

**多头注意力**（Multi-Head Attention）天然可以沿 head 维度切分： $W_Q, W_K, W_V$  列并行，每卡算一部分 head 的 QKV，Softmax、mask、点积都在卡内完成无需跨卡通信； $W_O$  行并行，把各 head 输出投影后求部分和，一次 Allreduce。

每个 Transformer block（attention + MLP）前向需要 2 次 Allreduce，反向再加 2 次。通信量正比于  $bsh$ （Allreduce 约为数据的 2 倍）。TP 让单卡算力降到  $\frac{1}{t}$ ，但增加了常数级通信，所以 TP 只在高带宽 NVLink 域内才划算。扩展：**Sequence Parallelism** 在 attention/MLP 外围插入 AllGather/Reduce-Scatter，进一步切分 LayerNorm/Dropout 的激活。



## 7 上下文并行与专家并行

### 7.1 上下文并行 (CP)

长序列 (32K、128K) 下, attention 的中间张量  $S, P \in R^{B \times N \times S \times S}$  单卡放不下。

**Sequence Parallelism** (Megatron) 在 TP 组内沿序列维切分 LayerNorm/Dropout 激活;

**Ring Attention** 把 KV 沿序列分到不同卡, 相邻卡组成环传递 KV 块:  $O(1)$  峰值显存换  $O(P)$  通信轮次, 配合 Flash Attention 可无损计算超长上下文。

核心难点在于  $S$  维: attention 的  $QK^T$  沿  $S$  归约, 切分后必须通信才能得到正确结果。这正是 Ring Attention / Sequence Parallelism 要解决的问题。

### 7.2 专家并行 (EP) 与 MoE

**Mixture of Experts** (MoE) 用 router 为每个 token 选 top-k 个专家, 只激活这些专家计算。参数量随专家数线性增长, 但计算量几乎不变, 提供了独立于层数  $L$  和隐藏维  $H$  的扩容维度。**DeepSeek-V3** 是 6710 亿参数、370 亿激活。

**专家并行** (GShard-MoE) 把不同专家放到不同 GPU 上, 用 all-to-all 派发 token。挑战是 router 输出不均衡导致专家负载不均 (引入辅助 loss 或 expert choice routing 强制均衡), 以及 all-to-all 对拓扑要求高。

## 8 6D 并行与自动并行

实际训练把六个维度组合起来。以 GPT-3 175B 为例: TP=8 (NVLink 内单节点 8 卡 A100), PP=8 (8 台机器一台一 stage), DP=60 (60 个 3D 并行副本), 共  $8 \times 8 \times 60 = 3840$  张 GPU。

如何自动选择切分方案? **Alpa** 分层优化: 算子间 stage 划分 (DP 最小化流水线延迟)、算子内切分 (ILP 为每个算子选策略)、设备映射 (把逻辑 mesh 映射到物理拓扑), 达到手调 Megatron 级性能且全自动。OneFlow SBP、PyTorch DTensor 是类似工业实现。

## 9 Transformer 定量分析

### 9.1 注意力与多头注意力

**Self-Attention** 用三个矩阵  $Q = XW_Q, K = XW_K, V = XW_V$ , 然后  $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ 。与 RNN 的关键区别是可并行, 不依赖上一步输出。

**多头注意力** (MHA) 让  $h$  个头各自处理  $\frac{H}{n_h}$  维, 拼接后还要一个  $H \times H$  的输出投影  $W_O$  映射回隐藏维。所以  $W_Q, W_K, W_V, W_O$  都是  $H \times H$ , 总参数  $4H^2$  (不是  $3H^2$ , 因为多了  $W_O$ )。

现代变体: **Pre-Norm** (残差前做 LayerNorm) 训练更稳; **RMSNorm** (LLaMA) 去掉中心化更快; **RoPE** 旋转位置编码解决外推; **SwiGLU** 给 FFN 加门控; **GQA/MQA** 让多个



Q 头共享 K/V，缩小 KV cache。但主要算力仍来自 Self-Attention 和 FFN，其余只是显存/带宽优化。

## 9.2 参数量与 FLOPs

LLaMA-2 7B 的结构：层数  $L = 32$ 、隐藏  $H = 4096$ 、中间  $I = 11008$ 、词表  $V = 32000$ 、头数  $n_h = 32$ 。参数量  $\approx L \cdot (4H^2 + 3HI) + V \cdot H$ ，其中  $4H^2$  是每层 attention， $3HI$  是每层 FFN (SwiGLU)， $VH$  是 embedding。代入得每层约 202M，32 层约 6.5B，加 embedding 共约 6.7B。

**例** 取短序列  $S \ll H$ ，主导项是  $8BSH^2 + 6BSHI \approx 8BSH^2(1 + \frac{3I}{4H})$ 。取长序列  $S \gg H$ ，attention 的  $S^2$  项  $4BS^2H$  成为瓶颈，这正是 Flash Attention 和 Ring Attention 要解决的。

FLOPs 估算前提：矩阵乘  $M_{m \times n} \cdot M_{n \times k}$  约需  $2mnk$  次浮点运算。

## 9.3 显存估算与梯度检查点

训练态显存 (Adam) 每个参数 16 字节 (FP16 权重 2 + FP32 master 4 + 梯度 2 + Adam  $\frac{m}{v}$  8)。以 7B 模型为例，参数约 112 GB；70B 模型约 1120 GB。一张 A100 (80GB) 勉强能训 7B，训 70B 必须用 ZeRO/TP/PP。

**梯度检查点** (Gradient Checkpointing) 不保存中间激活，反向时重算前向，把激活显存降到  $O(\sqrt{L})$ ，代价约多 25% 计算。

# 10 大模型推理系统

## 10.1 推理的两阶段

LLM 推理分两阶段：**Prefill** (首字阶段) 一次性处理所有输入 token，是计算密集型，能把 GPU 打满；**Decode** (解码阶段) 每步只生成 1 个新 token，是访存带宽密集型，GPU 严重欠载。**KV cache** 是解码阶段性能和显存的关键。

**直觉** Prefill 像训练的前向， $S = 1024$  一次性算，GPU 忙不过来；Decode 每步  $S = 1$ ，算力远没用满，瓶颈变成把 KV cache 从显存搬到计算单元的带宽。

## 10.2 KV Cache

**KV Cache** 在每个解码步保存 attention 的 key/value，避免重算历史 token 的 K/V，用空间换时间。大小  $= 2 \cdot L \cdot n_h \cdot d \cdot S \cdot B \cdot \text{sizeof}(\text{dtype})$ 。

**例** LLaMA-2 7B， $S = 2048$ ， $B = 1$ ，FP16: KV cache 约 1 GB。

为什么不缓存 Q? 因为 Q 是当前新 token 的查询向量，每步都不同，没有复用价值；K/V 是历史向量，跨步复用，才值得缓存。

## 10.3 连续批处理

传统请求级调度让一批请求一起进 GPU，最短的等最长的，新请求等整批结束，GPU 大量空闲。迭代级调度（Continuous Batching, Orca/vLLM）每生成一个 token 就重组一次 batch，请求可随时加入和退出。

影响：TTFT（首字延迟）由 *prefill* 排队 + *prefill* 计算决定，连续批处理让新请求不用等整批，TTFT 大幅下降，但若与解码 *batch* 抢算力会抬高 TPOT。TPOT（每 token 延迟）由解码阶段每迭代批大小和访存决定，*batch* 越满吞吐越高，但混入 *prefill* 会造成周期性抖动（*chunked prefill* 可缓解）。实测比静态批处理吞吐提升最高约 20 倍。

## 10.4 Prefill/Decode 分离部署

连续批处理的问题是 Prefill（计算密集）和 Decode（访存密集）在同一张卡上抢资源，造成 TTFT 抖动。分离部署把 Prefill 和 Decode 放到不同 GPU 池，各自独立扩缩，需要高效的 KV 迁移（NVLink/RDMA）。代表系统有 DistServe、Splitwise、DeepSpeed-Inference。

## 10.5 Paged Attention

传统 KV cache 有浪费：为最坏情况预留内存（Reservation），过度预留造成内部碎片，并发请求造成外部碎片。Paged Attention 借鉴操作系统的虚拟内存和分页机制，把 KV cache 按页管理，按需分配，消除预留和碎片，吞吐提升约 3 倍。

# 11 Flash Attention

### 11.1 动机：S 维爆炸

标准 attention 实现必须显式物化  $S = QK^T \in R^{B \times N \times S \times S}$ 。当  $S = 32K$  时，单个头的  $S$  矩阵就有  $32^2 M = 1 GB$ 。反复在 HBM 和 SRAM 之间搬运这两个大矩阵  $S, P$  产生巨大通信开销。

核心问题是：能否避免物化  $S$ ，直接得到 attention 输出？答案是可以，关键在于巧妙组织 softmax + 矩阵乘的顺序，让所有中间结果都留在 SRAM 里。

收益：避免物化  $bs^2n$  的  $S$  矩阵，大幅减少 HBM 与 SRAM 间的数据搬运；代价是略微增加计算（rescale 和重读 K/V）。整体训练加速 2 到 4 倍，推理加速取决于  $S$ 。下游影响：让  $S = 32 \frac{K}{128} K$  训练成为可能，可省去梯度检查点（省约 25% 计算）。v2/v3 进一步利用 warp 级并行、TMA、异步 WGMMMA；配合 Ring Attention 支持无限上下文。

# 12 前沿与总结

### 12.1 投机解码

解码阶段  $B = 1$  时 GPU 严重欠载，投机解码（Speculative Decoding）用小模型猜接下来  $k$  个 token，再用大模型一次前向验证，加速正比于平均接受长度（通常 2 到 4 倍）。变体有 Medusa（多头并行猜）、Eagle（基于特征起草）。DeepSeek-V3 的 MTP（Multi-Token Prediction）在训练时就学习多个未来 token。

## 12.2 DeepSeek-V4: CSA + HCA 双流混合

2026 年的 DeepSeek-V4 规模达 1.6T 总参数 / 49B 激活 (MoE)，原生 100 万 token 上下文。它把两种 attention 按 1:1 交替堆叠：**CSA**（压缩稀疏注意力）先压缩 KV 再 top-k 稀疏，保留细节；**HCA**（重度压缩注意力）更激进地压缩，提供粗糙全局视图。V3 的 MLA 只压缩 KV，V4 用 CSA + HCA 交替，在百万上下文下兼顾细节与全局。

## 12.3 硬件视角：放什么在哪

并行的选择取决于硬件拓扑：

1. **GPU 内**：HBM 带宽 3 到 4 TB/s，延迟 100 ns，放计算和 Tile。
2. **节点内 (NVLink)**：带宽 900 GB/s (NVLink4)，延迟约 1  $\mu$ s，放 Tensor Parallelism。
3. **跨节点 (InfiniBand)**：400 Gbps (NDR)，延迟约 5  $\mu$ s，放 Pipeline/Data Parallelism。

规则：TP 每层多次 Allreduce，必须留在 NVLink 域内；PP 每 stage 只需 P2P，可跨机器；DP 每迭代一次梯度 Allreduce，跨机器代价可接受。DGX SuperPOD 用 NVLink Switch 把 32 节点 8 卡 H100 连成一张大网。

## 13 本章你将学会

1. 解释六种并行维度 (DP/TP/PP/SP/CP/EP) 各自切分什么、属于算子间还是算子内，以及典型通信原语。
2. 用  $\alpha + n\beta$  模型分析 Ring Allreduce 为何带宽几乎与节点数无关，并解释通信与计算重叠的 bucketing 策略。
3. 推导 ZeRO 三个级别的显存节省和通信代价，说明 ZeRO-3 为何是“用带宽换显存”。
4. 描述 GPipe 与 1F1B 的气泡率和峰值显存差异，以及 Megatron-LM 列并行  $\rightarrow$  行并行为何只需一次 Allreduce。
5. 估算 Transformer 的参数量 ( $4H^2 + 3HI$ ) 和 FLOPs ( $8BSH^2 + 4BS^2H + 6BSHI$ )，判断短/长序列下的算力瓶颈。
6. 解释 Prefill 与 Decode 的本质区别、KV cache 的作用、连续批处理与 Paged Attention 如何提升吞吐。
7. 说明 Flash Attention 为何能避免物化  $S$  矩阵及其对长上下文训练的意义。

## 14 要点速查

| 概念    | 英文 | 一句话要点                       |
|-------|----|-----------------------------|
| 数据并行  | DP | 切 batch，每卡完整模型，Allreduce 梯度 |
| 流水线并行 | PP | 切层到不同卡，算子间，P2P 通信           |
| 张量并行  | TP | 切层内张量，算子内，Allreduce 通信      |

|                 |                     |                                                  |
|-----------------|---------------------|--------------------------------------------------|
| ZeRO-3/FSDP     | FSDP                | 切参数+梯度+优化器，显存 $\frac{1}{N}$ ，通信 3 倍              |
| Ring Allreduce  | Ring Allreduce      | 带宽项趋近 $n\beta$ ，与节点数无关                           |
| GPipe           | GPipe               | 切微批次，气泡率 $\frac{D-1}{D-1+N}$ ，显存 $O(N)$          |
| 1F1B            | 1F1B                | 尽早反向，峰值显存降到 $O(D)$                               |
| Megatron-LM     | Megatron-LM         | 列并行到行并行，一个 MLP 只需一次 Allreduce                    |
| Ring Attention  | CP                  | 沿序列切 KV，环形传递，支持超长上下文                             |
| MoE/EP          | 专家并行                | router 选 top-k 专家，all-to-all 派发 token            |
| KV Cache        | KV Cache            | 缓存历史 K/V 换时间，不缓存 Q                               |
| 连续批处理           | Continuous Batching | 迭代级调度，吞吐最高约 20 倍                                 |
| Paged Attention | Paged Attention     | 按页管理 KV cache，消除碎片，吞吐约 3 倍                       |
| Flash Attention | Flash Attention     | 不物化 $S$ ，融合 online softmax + tiling，训练加速 2 到 4 倍 |

## 15 小结

本章从“单卡放不下大模型”这一现实出发，系统梳理了分布式训练的六种并行维度和通信基础。我们看到了通信代价模型  $\alpha + n\beta$  如何决定算法选择：小消息用 MST、大消息用 Ring，而通信与计算重叠是所有优化的共同底色。数据并行家族从参数服务器演进到 Allreduce (DDP)，再到 ZeRO 1/2/3 和 FSDP，用带宽换显存；流水线家族从 GPipe 到 1F1B 再到交错调度，把气泡和峰值显存逐步压低；张量并行家族从单算子切分到 Megatron-LM 再到 Alpa 自动并行。

在大模型系统部分，我们用 Transformer 的定量分析看清了算力和显存的去向：短序列瓶颈在 MLP，长序列瓶颈在 attention 的  $S^2$  项。推理系统的两阶段本质（Prefill 计算密集、Decode 访存密集）催生了 KV cache、连续批处理、Prefill/Decode 分离和 Paged Attention 等优化。Flash Attention 通过不物化  $S$  矩阵让长上下文训练成为可能。2026 年的前沿（DeepSeek-V4 的 CSA+HCA、投机解码、MTP）则在算法层面继续突破  $O(n^2)$  的束缚。

一句话总结：分布式系统 + 硬件感知 + 算法创新 = 现代 LLM 的基石。

讲义基于 HPC101 课程「Distributed Training & Large-Model Systems」内容编写